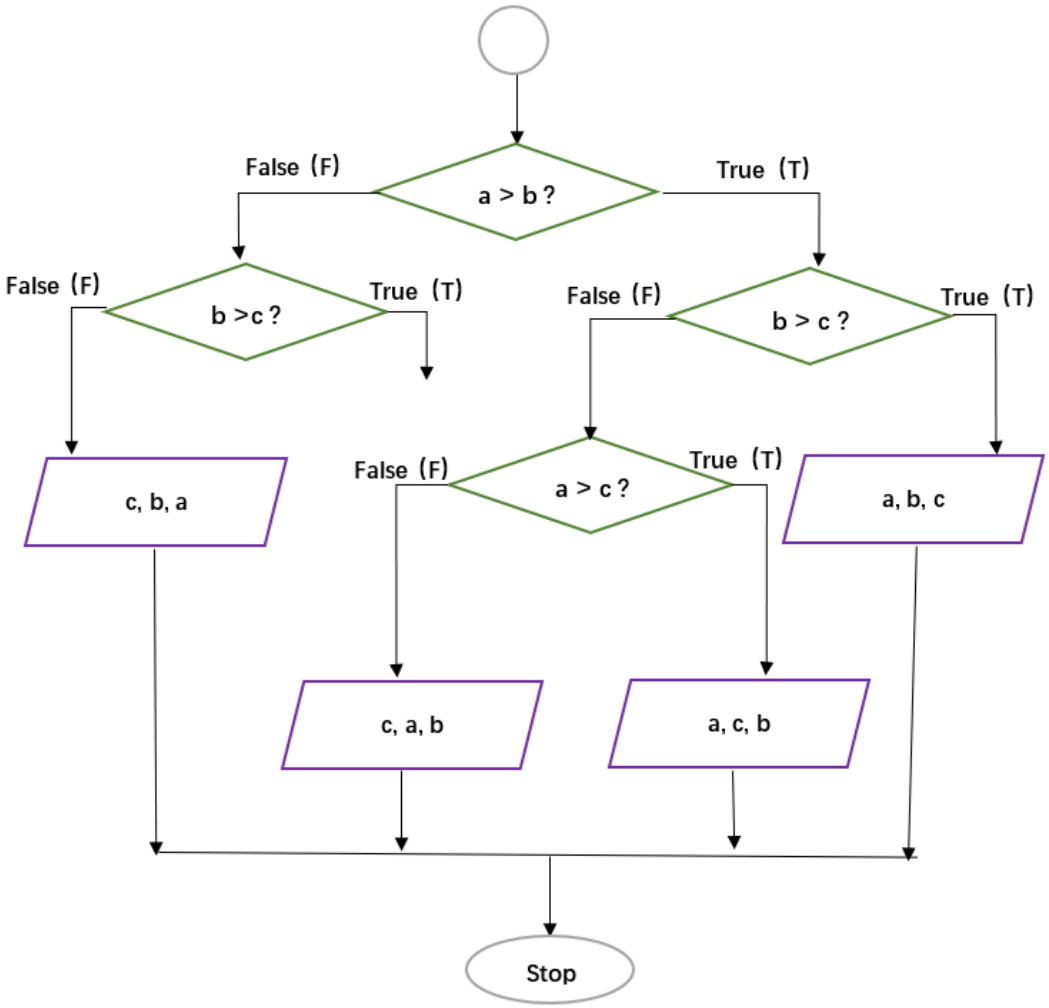
1. Flowchart

[10 points] Write a function Print_values with arguments a, b, and c to reflect the following flowchart. Here the purple parallelogram operator is to print values in the given order. Report your output with some random a, b, and c values.




```
#PS1_1 definition
#True judgement prior, False judgement belongs to "else"
#so that when one element equals to another, it still make sense
def Print_values(a, b, c):
    if a > b :
        if b > c :
            print(a, b, c)
        else :
            if a > c :
                print(a, c, b)
```

```
            else :
                print(c, a, b)
    else:
        if b > c:
            if a > c :
                print(a, c, b)
            else :
                print(c, a, b)
        else:
            print(c, b, a)
```

```
#PS1_1 report
a = float(input("please enter value a = "))
b = float(input("please enter value b = "))
c = float(input("please enter value c = "))
Print_values(a, b, c)
```

```
please enter value a =  100
please enter value b =  1000
please enter value c =  10
100.0 10.0 1000.0
```

2. Matrix multiplication

2.1 [5 points] Make two matrices M1 (5 rows and 10 columns) and M2 (10 rows and 5 columns); both are filled with random integers from 0 and 50.

2.2 [10 points] Write a function Matrix_multip to do matrix multiplication, *i.e.*, M1 * M2. Here you are **ONLY** allowed to use for loop, * operator, and + operator

```
#PS1_2 definition

def Matrix_multip(M1,M2):
    multi = np.full((5,5),0)
    #define an original result array with full values of 0
    for i in range(5):
        for j in range(5):
            number = 0
            for k in range(10):
                number = number + M1[i][k]*M2[k][j]
            multi[i][j] = number
    #for M1: i rows and k column; for M2: k rows and j column
    #for multi: o was replaced before next loop of "number = 0"
    print("the function result is: \n", multi)
    return multi

#PS1_2 output
import numpy as np
#creat array in specific shape with random integer between 0-50(51 is not included)
M1 = np.random.randint(0,51,size = (5,10))
M2 = np.random.randint(0,51,size = (10,5))
print("M1: \n",M1)
print("M2: \n",M2)
multi = Matrix_multip(M1,M2)
result = M1 @ M2
print("the correct answer is: \n", result)
print("the authentication is: \n",np.array_equal(multi, M1 @ M2))
## the authentication function is informed by Codex
```

```
M1:
 [[16  4 30 14 15 35 31 26 17 25]
 [ 6 10 48 28 43 49 22 26 50 27]
```

[0 37 13 13 4 41 30 26 9 21]
[23 41 7 26 34 22 14 27 41 35]
[17 4 24 23 2 3 23 11 46 46]]
M2:
[[44 26 6 40 2]
[50 23 40 34 46]
[18 32 37 45 6]
[29 26 35 17 13]
[22 3 17 6 30]
[40 24 39 31 40]
[41 24 24 19 14]
[39 11 22 27 21]
[46 15 47 12 36]
[29 40 8 32 48]]
the function result is:
[[7372 5002 5791 5834 5220]
[10345 6599 9500 7577 8324]
[7456 4582 5966 5411 6007]
[10098 5719 7520 6564 8131]
[7033 5181 5430 5150 5258]]
the correct answer is:
[[7372 5002 5791 5834 5220]
[10345 6599 9500 7577 8324]
[7456 4582 5966 5411 6007]
[10098 5719 7520 6564 8131]
[7033 5181 5430 5150 5258]]
the authentication is:
True

3. Pascal triangle

[20 points] One of the most interesting number patterns is Pascal's triangle (named after Blaise Pascal). Write a function Pascal_triangle with an argument k to print the kth line of the Pascal triangle. Report Pascal_triangle(100) and Pascal_triangle(200)

```
#PS1_3 definition
def Pascal_triangle(k):
    row = [1]
        ##the first number is 1,
    ##so the calculation start with the second number
    for lines in range(2,k+1):
        new_row = [1]
        ##the last number should be 1
        ##so end with the one before it
        for i in range(1,lines-1):
            number = row[i-1] + row[i]
            new_row.append(number)
        new_row.append(1)
        ##to append the last number of new_row
        row = new_row
        ##loop to the next line

    print("the ", k,"^th line is:")
    print(row)

    summary = 0
    ##sequence start with 0
    for i in range(0,k):
        summary = summary + row[i]
   print("the summary of ", k,"^th line is", summary)

#PS1_3 report
k = int(input("Please enter k: "))
Pascal_triangle(k)
Pascal_triangle(100)
Pascal_triangle(200)
Please enter k:  15
the  15 ^th line is:
```

[1, 14, 91, 364, 1001, 2002, 3003, 3432, 3003, 2002, 1001, 364, 91, 14, 1]
the summary of 15 ^th line is 16384
the 100 ^th line is:
[1, 99, 4851, 156849, 3764376, 71523144, 1120529256, 14887031544, 171200862756,
1731030945644, 15579278510796, 126050526132804, 924370524973896,
6186171974825304, 38000770702498296, 215337700647490344,
1130522928399324306, 5519611944537877494, 25144898858450330806,
107196674080761936594, 428786696323047746376, 161305471473908437922 4,
5719012170438571889976, 19146258135816088501224, 60629817430084280253876,
18188945229025284076162 8, 51768536421071962370617 2,
13996678365697234270574 28, 35991458654650030981476 72,
88117019464832834471891 28, 20560637875127661376774632,
45764000431735762419272568, 97248500917438495140954207,
19744392610510239922557 3693, 38327350361578701026140 7757,
71179364957217587619975 7263, 12654109325727571132440 12912,
21546186149211810306587 24688, 35154303717135058921273 92912,
54984936583211246005069 47888, 82477404874816869007604 21832,
11868699725888281149874753368, 16390109145274293016493707032,
21726423750712434928840495368, 27651812046361280818524266832,
33796659167774898778196326128, 39674339023040098565708730672,
44739148260023940935799206928, 48467410615025936013782474172,
50445672272782096667406248628, 50445672272782096667406248628,
48467410615025936013782474172, 44739148260023940935799206928,
39674339023040098565708730672, 33796659167774898778196326128,
27651812046361280818524266832, 21726423750712434928840495368,
16390109145274293016493707032, 11868699725888281149874753368,
82477404874816869007604 21832, 54984936583211246005069 47888,
35154303717135058921273 92912, 21546186149211810306587 24688,
12654109325727571132440 12912, 71179364957217587619975 7263,
38327350361578701026140 7757, 19744392610510239922557 3693,
97248500917438495140954207, 45764000431735762419272568,
20560637875127661376774632, 88117019464832834471891 28,
35991458654650030981476 72, 13996678365697234270574 28,
51768536421071962370617 2, 18188945229025284076162 8,
60629817430084280253876, 19146258135816088501224, 57190121704385718899 76,
16130547147390843792 24, 42878669632304774637 6, 10719667408076193659 4,
25144898858450330806, 55196119445378774 94, 11305229283993243 06,
215337700647490344, 38000770702498296, 6186171974825304, 924370524973896,

126050526132804, 15579278510796, 1731030945644, 171200862756, 14887031544,
1120529256, 71523144, 3764376, 156849, 4851, 99, 1]
the summary of 100 ^th line is 6338253001141147007483516026 88
the 200 ^th line is:
[1, 199, 19701, 1293699, 63391251, 2472258789, 79936367511, 2203959847089,
52895036330136, 1122550215450664, 21328454093562616, 366461620334848584,
5741232051912627816, 82585414900589338584, 1097206226536401212616,
13532210127282281622264, 155620416463746238656036,
16752080125215036278855 64, 16938214348828536681954036,
16135877879673500733861 4764, 14522290091706150660475328 76,
12378523459120956991548018324, 10015350798743319747707033 0076,
77074656146850765014962819 2324, 56521414507690561010972734 10376,
39564990155383392707680913872632, 26478108796295039735140303899 3768,
16965603043552006941404713239230 32, 10421727583896232835434323846955768,
61452255753319166029629978545842632,
34822944926880860750123654509310 8248,
18984121589170533763777089071204 93352,
99666638343145302259829717623825 90098,
50437359403955349931489584373269471102,
24625299003107612025374326488125 6829498,
11609069530036445669105039630116393390 62,
52885761192388252492589624981641347668 38,
23298321822592662584573267221641999107962,
99324424612105561544759718155421154091838,
41003159903971783099246960571853348227 6562,
16401263961588713239698784228741339291 06248,
63604901704697692807612358350484706031 19352,
23927558260338655865720839569944246554591848,
87363410392399278393445856104215039745835352,
30974300048214289612221712618767150455341 6248,
10668925572162699755320812124242018490173 22632,
35717707350283820919987066676810235814927 75768,
11627253669347711916506428088408438467412 653032,
36819636619601087735603688946626721813473 401268,
11346459448081151526686034757021704069049 9665132,
34039378344243454580058104271065112207149 8995396,
99448379868475975145659951693896112134614 4123804,
28304538885643162156841678559031970376774 87121596,

78505041814897072397277863173164144252564265448 04,
21225437231435134388893644487559194557174782880396,
55957970882874445207083244558110603832551700321044,
14389192512739143053249977172085583842656151511 1256,
36099202268801709765170995361548043675435608177 0344,
88380805554652461838866919678272796584687178640 3256,
21121514547806774778441077414638075116001512183 53544,
49283533944882474483029180634155508604003528428 24936,
11230182325145350742854190341225599501568017133 650264,
24996212272097716169578681727244076309941715555 544136,
54356842559958525638607609470356165943841508430 310264,
11550829043991186698204117012450685263066320541 4409311,
23990183399058618527039319948936038623291588816 8388569,
48707342052634164888231346562991351144258680325 0970731,
96687708850751401942309986460863428390841857958 7747869,
18768790541616448612330762077697018452339890074 35039981,
35633500883358764756743910611279846626906168112 17249819,
66176501640523420262524405420948286592825740779 74892521,
12023617903700734104036124365214547845738761352 940297679,
21375320717690193962730887760381418392424464627 449418096,
37187201796529515524203051309156714189560369968 302412304,
63318749004901607514183573850726297133575765081 163566896,
10553124834150267919030595641787716188929294180 1939278160,
17218256308350437131049919205022063255621479978 2111453840,
27504487349702646326222598210619659486252493991 1684530160,
43019839187996446817937910021738441760548772652 8532213840,
65891146098070507151525153324434828519321537860 6992378160,
98836719147105760727287729986652242778982306791 0488567240,
14520456269759982131539802306681008507035672232 26520240760,
20895290729654608433191422831565353705246455163 50358395240,
29454807414091435984137306883049956427877533182 28818460760,
40675686428983411597141995219449939828973736299 35035017240,
55031811050977556866721522943961683298023290287 35635611560,
72949144881528389334956437390832929022961105729 75144880440,
94750038754169057412069855461656562983846033878 87257143560,
12059095841439698216081617967847198925216767948 220145455440,
15039995937076477550393928027315045850551249912 948720736560,
18382217256426805894925912033385056039562638782 492880900240,

220182602302255147532629056224062759155200838163925716277 60,
258475228789603868842651500784769325964800983931564971282 40,
297385478284813053399609791225487289013265648179327440077 60,
335349581895640251705942956062783538674533603266050092002 40,
370649537884655015043410635648339700640273982557213259581 60,
401536999375042932963694855285701342360296814436981031213 40,
426374339542571361806810000973476683124851256567103569226 60,
443777373809615090860149184686679813048314573161679225113 40,
452742573280516405827020885387420819372522948377066684206 60,
452742573280516405827020885387420819372522948377066684206 60,
443777373809615090860149184686679813048314573161679225113 40,
426374339542571361806810000973476683124851256567103569226 60,
401536999375042932963694855285701342360296814436981031213 40,
370649537884655015043410635648339700640273982557213259581 60,
335349581895640251705942956062783538674533603266050092002 40,
297385478284813053399609791225487289013265648179327440077 60,
258475228789603868842651500784769325964800983931564971282 40,
220182602302255147532629056224062759155200838163925716277 60,
183822172564268058949259120333850560395626387824928809002 40,
150399959370764775503939280273150458505512499129487207365 60,
120590958414396982160816179678471989252167679482201454554 40,
947500387541690574120698554616565629838460338788725714356 0,
729491448815283893349564373908329290229611057297514488044 0,
550318110509775568667215229439616832980232902873563561156 0,
406756864289834115971419952194499398289737362993503501724 0,
294548074140914359841373068830499564278775331822881846076 0,
208952907296546084331914228315653537052464551635035839524 0,
145204562697599821315398023066810085070356722322652024076 0,
988367191471057607272877299866522427789823067910488567240,
658911460980705071515251533244348285193215378606992378160,
430198391879964468179379100217384417605487726528532213840,
275044873497026463262225982106196594862524939911684530160,
172182563083504371310499192050220632556214799782111453840,
105531248341502679190305956417877161889292941801939278160,
633187490049016075141835738507262971335757650811635668 96,
371872017965295155242030513091567141895603699683024123 04,
213753207176901939627308877603814183924244646274494180 96,
120236179037007341040361243652145478457387613529402976 79,

661765016405234202625244054209482865928257407797489252 1,
356335008833587647567439106112798466269061681121724981 9,
187687905416164486123307620776970184523398900743503998 1,
966877088507514019423099864608634283908418579587747869,
487073420526341648882313465629913511442586803250970731,
239901833990586185270393199489360386232915888168388569,
115508290439911866982041170124506852630663205414409311,
543568425599585256386076094703561659438415084303102 64,
249962122720977161695786817272440763099417155555441 36,
112301823251453507428541903412255995015680171336502 64,
492835339448824744830291806341555086040035284282493 6,
211215145478067747784410774146380751160015121835354 4,
883808055546524618388669196782727965846871786403256,
360992022688017097651709953615480436754356081770344,
143891925127391430532499771720855838426561515111256,
559579708828744452070832445581106038325517003210 44,
212254372314351343888936444875591945571747828803 96,
785050418148970723972778631731641442525642654480 4,
283045388856431621568416785590319703767748712159 6,
994483798684759751456599516938961121346144123804,
340393783442434545800581042710651122071498995396,
113464594480811515266860347570217040690499665132,
368196366196010877356036889466267218134734012 68,
116272536693477119165064280884084384674126530 32,
357177073502838209199870666768102358149277576 8,
106689255721626997553208121242420184901732263 2,
309743000482142896122217126187671504553416248,
873634103923992783934458561042150397458353 52,
239275582603386558657208395699442465545918 48,
636049017046976928076123583504847060311935 2,
164012639615887132396987842287413392910624 8,
410031599039717830992469605718533482276562,
993244246121055615447597181554211540918 38,
232983218225926625845732672216419991079 62,
528857611923882524925896249816413476683 8,
116090695300364456691050396301163933906 2,
246252990031076120253743264881256829498,
504373594039553499314895843732694711 02,

99666638343145302259829717623825900 98,
18984121589170533763777089071204933 52,
34822944926880860750123654509310824 8,
61452255753319166029629978545842632, 10421727583896232835434323846955768,
16965603043552006941404713239230 32, 26478108796295039735140303899376 8,
39564990155383392707680913872632, 56521414507690561010972734103 76,
77074656146850765014962819232 4, 10015350798743319747707033007 6,
12378523459120956991548018324, 14522290091706150660475328 76,
16135877879673500733861476 4, 16938214348828536681954036,
16752080125215036278855 64, 15562041646374623865603 6,
13532210127282281622264, 10972062265364012126 16, 82585414900589338584,
5741232051912627816, 366461620334848584, 21328454093562616,
1122550215450664, 52895036330136, 2203959847089, 79936367511, 2472258789,
63391251, 1293699, 19701, 199, 1]
the summary of 200 ^th line is
80346902212949513777098104617058130126110149689139641765 0688

4. Add or double

[20 points] If you start with 1 RMB and, with each move, you can either double your money or add another 1 RMB, what is the smallest number of moves you have to make to get to exactly *x* RMB? Here *x* is an integer randomly selected from 1 to 100. Write a function Least_moves to print your results. For example, Least_moves(2) should print 1, and Least_moves(5) should print 3

```
#PS1_4 definition
#for even number, the last step should be double
#for odd number, the last step should be plus
#during the moves, double step must be faster than plus
def Least_moves(x):

    moves = 0
    while x > 1 :
        if x % 2 == 0:
            x = x // 2
            ##double // to maitain integer type
        else:
            x = x-1
        moves = moves + 1
    return moves

#PS1_4 output
x = int(input("how much money you want to get(<100)?"))
result = Least_moves(x)
print("the least moves to", x ,"is ", result)
```

```
how much money you want to get(<100)? 5
the least moves to 5 is  3
```

5. Dynamic programming

Insert + or - operation anywhere between the digits 123456789 in a way that the expression evaluates to an integer number. You may join digits together to form a bigger number. However, the digits must stay in the original order.

5.1 [30 points] Write a function Find_expression, which should be able to print every possible solution that makes the expression evaluate to a random integer from 1 to 100. For example, Find_expression(50) should print lines include:

5.2 [5 points] Count the total number of suitable solutions for any integer *i* from 1 to 100, assign the count to a list called Total_solutions. Plot the list Total_solutions, so which number(s) yields the maximum and minimum of Total_solutions?

```
#PS1_5.1    definition
def Find_expression(x):
    digits = "123456789"
    expression_list = ["1"]
    ##start with digit "1"
    choices = ["","+","-"]

    ##creat a list with all expressions
    for i in range(1,9):
        ## loop for each digit
        new_expression_list = []
        ##record each expression

        for expre in expression_list:
            ##"expre" means the currently stored expressions, e.g. 12+3 and 12-3
and 123
                for symbol in choices:
                    newexpre = (expre + symbol + digits[i])
                    ##this line can concatenate current expres with next digit
                    new_expression_list.append(newexpre)

        expression_list =    new_expression_list

    #creat a dictionary with value calculated by all expressions
    solution = {}
    for expre in expression_list:
        value = eval(expre)
```

```
            ## eval() get the value of a string
            if value not in solution:
                solution[value] = []
            solution[value].append(expre)
            #sort the different expressions of same value
    ##Find all results of the target number
    results = solution.get(x,[])
    ##create a list "results" to stored the selected solutions
    if x not in solution:
        print("target number've got not solution, target number:", x)
    else:
        for expre in results:
            print(expre, "=", x)
    return results



##PS1_5.1 output
x = int(input("please input a target number:"))

#Question: why would be a different output between these two lines?
results = Find_expression(x)
Find_expression(x)

please input a target number: 28
123-45-67+8+9 = 28
12+3+4+5-6-7+8+9 = 28
12+3+4-5+6+7-8+9 = 28
12+3-4+5+6+7+8-9 = 28
12+3-4-5-67+89 = 28
12-34+56-7-8+9 = 28
12-3-4+5-6+7+8+9 = 28
1+2+34+5-6-7+8-9 = 28
1+2+34-5+6+7-8-9 = 28
1+2+3-4-56-7+89 = 28
1+2-3-4+56-7-8-9 = 28
1-23+4+56+7-8-9 = 28
1-2+34-5+6-7-8+9 = 28
```

```
1-2+34-5-6+7+8-9 = 28
1-2-34-5+67-8+9 = 28
123-45-67+8+9 = 28
12+3+4+5-6-7+8+9 = 28
12+3+4-5+6+7-8+9 = 28
12+3-4+5+6+7+8-9 = 28
12+3-4-5-67+89 = 28
12-34+56-7-8+9 = 28
12-3-4+5-6+7+8+9 = 28
1+2+34+5-6-7+8-9 = 28
1+2+34-5+6+7-8-9 = 28
1+2+3-4-56-7+89 = 28
1+2-3-4+56-7-8-9 = 28
1-23+4+56+7-8-9 = 28
1-2+34-5+6-7-8+9 = 28
1-2+34-5-6+7+8-9 = 28
1-2-34-5+67-8+9 = 28
['123-45-67+8+9',
 '12+3+4+5-6-7+8+9',
 '12+3+4-5+6+7-8+9',
 '12+3-4+5+6+7+8-9',
 '12+3-4-5-67+89',
 '12-34+56-7-8+9',
 '12-3-4+5-6+7+8+9',
 '1+2+34+5-6-7+8-9',
 '1+2+34-5+6+7-8-9',
 '1+2+3-4-56-7+89',
 '1+2-3-4+56-7-8-9',
 '1-23+4+56+7-8-9',
 '1-2+34-5+6-7-8+9',
 '1-2+34-5-6+7+8-9',
 '1-2-34-5+67-8+9']
```

```
##PS1_5.2 ploting
import matplotlib.pyplot as plt
Total_solutions = []
for i in range(1,101):
        results = solution.get(i,[])
```

```
        frequency = len(results)

        Total_solutions.append(frequency)
number = range(1,101)
plt.plot(number, Total_solutions)

# Add x and y labels
plt.xlabel("number")
plt.ylabel("Total_solutions")

# Add figure title
plt.title("Total_solutions for numbers range from 1 to 100")

# Show plot
plt.show
```

```
<function matplotlib.pyplot.show(close=None, block=None)>
```

